

SchuLyf

A Student Management System for the university

**Digitising admissions, tamper-proof payments, exam-gating,
course management & verifiable academic transcripts**

PRESENTED BY

«Your Name»

INSTITUTION

«University / Faculty»

DEPARTMENT / COURSE

«Department · Course code»

SUPERVISOR

«Supervisor name»

DATE

«Month Year»

The problem

01

Manual admission

Paper forms carried to the office; slow, opaque, lossy.

02

Lost bank slips

A misplaced deposit slip blocks receipt issuance entirely.

03

Forgeable receipts

The paper school receipt can be tampered with or reused.

04

Payment disputes

Students and accountants clash over who can see bank records.

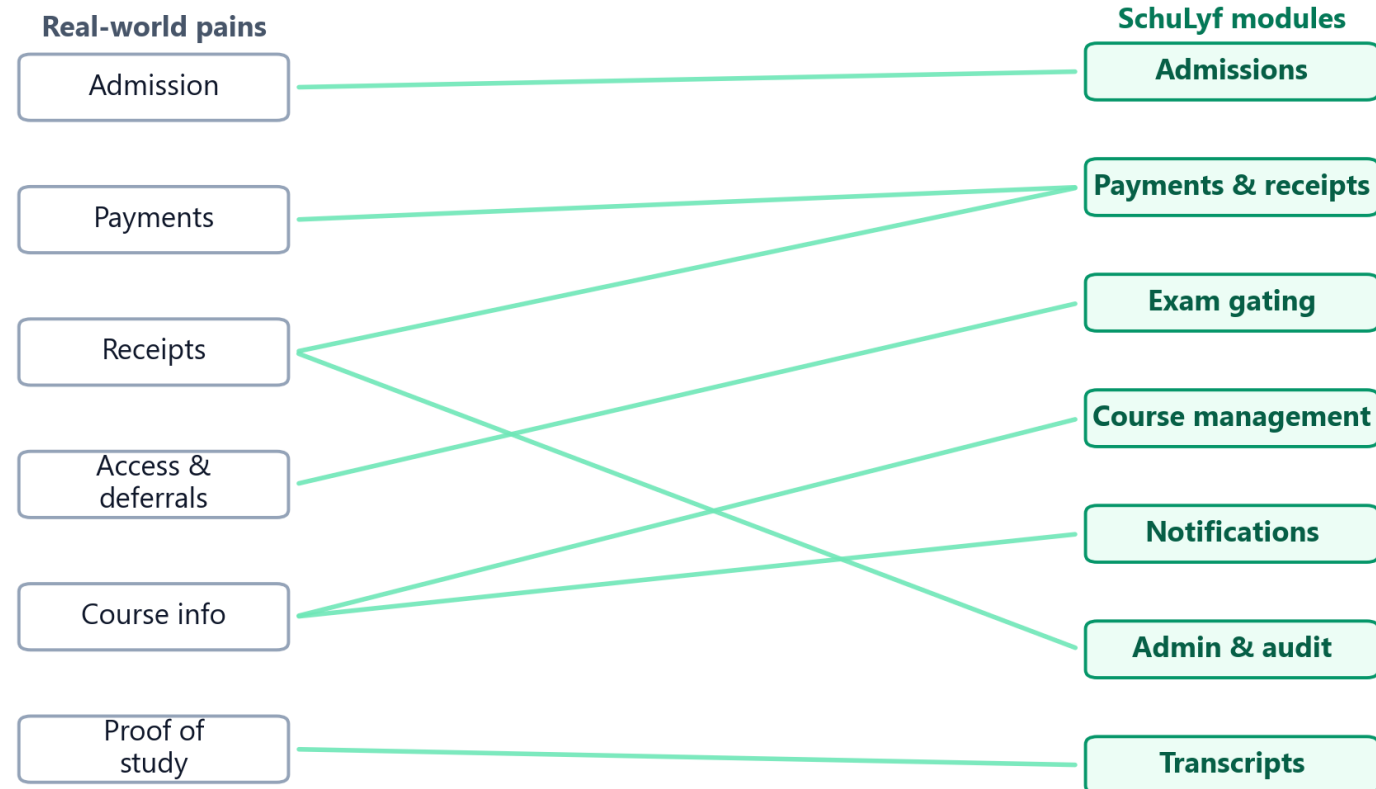
05

Exam-access rows

Standing at each deadline decides hall access — often contested.

Five recurring failures in day-to-day student administration.

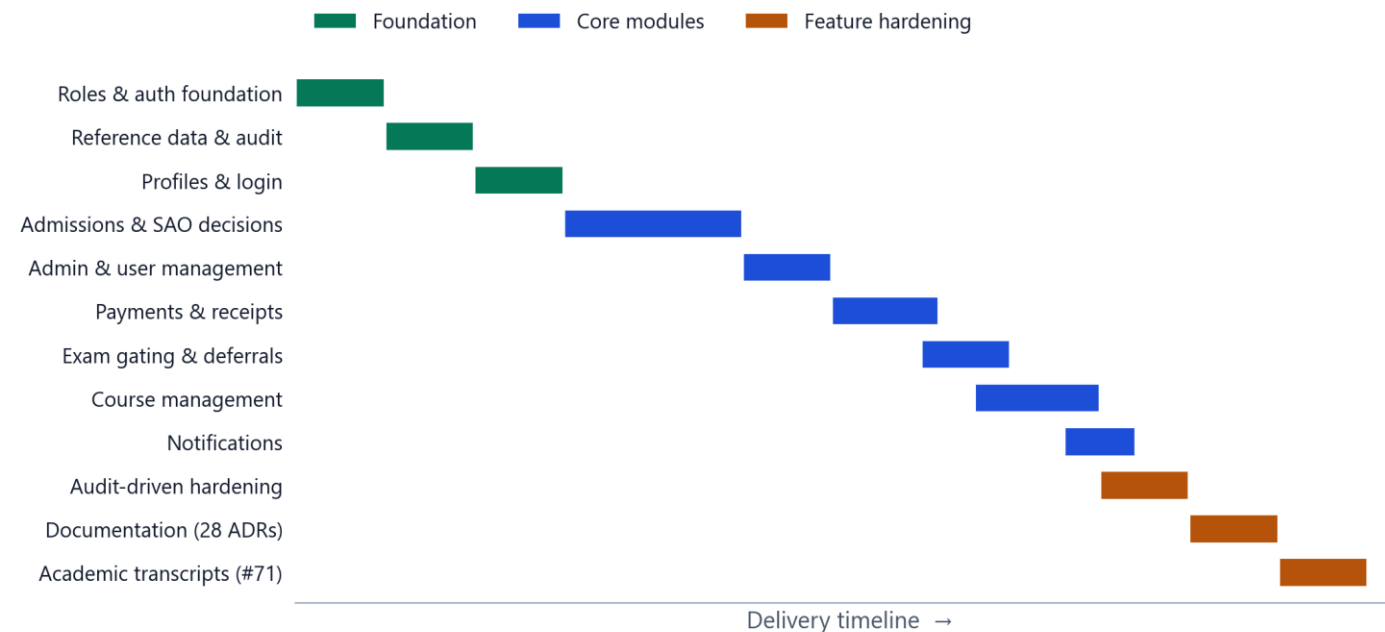
Objectives & scope



Each real-world pain maps to a dedicated SchuLyf module.

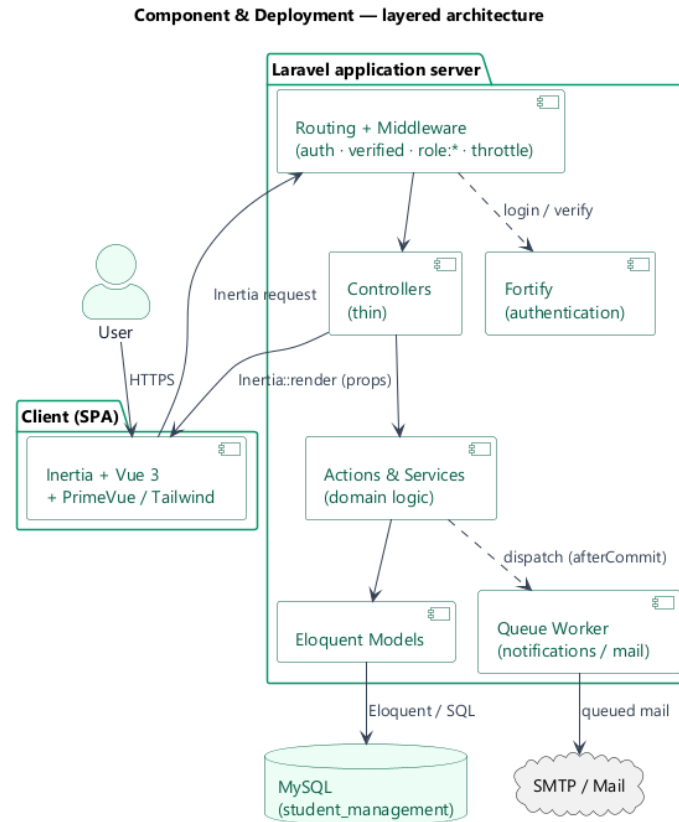
APPROACH

Methodology & delivery



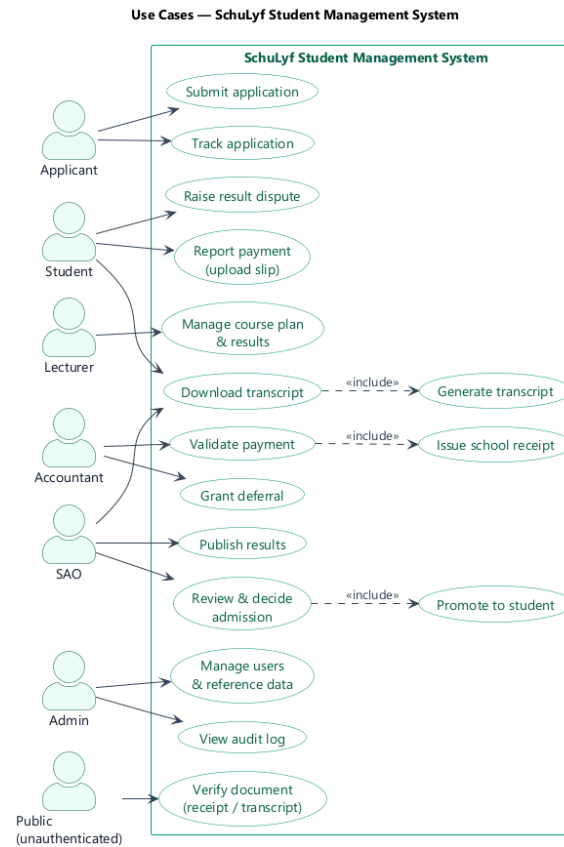
- **Laravel 13 + Inertia/Vue 3** full-stack, session-authenticated
- **Test-driven (Pest)** every change proven by a test
- **Per-feature quality gate** Pint · types · lint · build
- **Audit-driven hardening** 34 findings remediated
- **Phased, incremental delivery**

System architecture



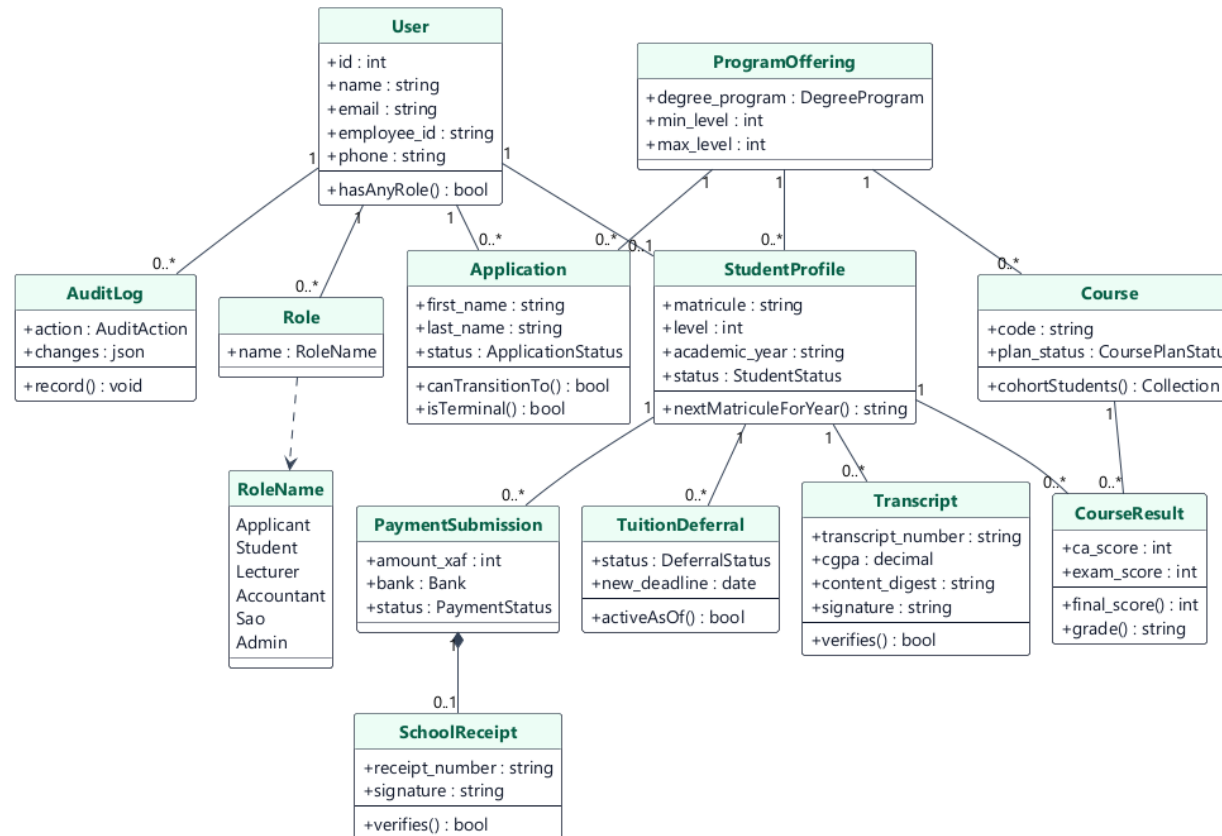
Thin controllers → actions/services → Eloquent → MySQL; queued mail; Fortify auth.

Actors & use cases



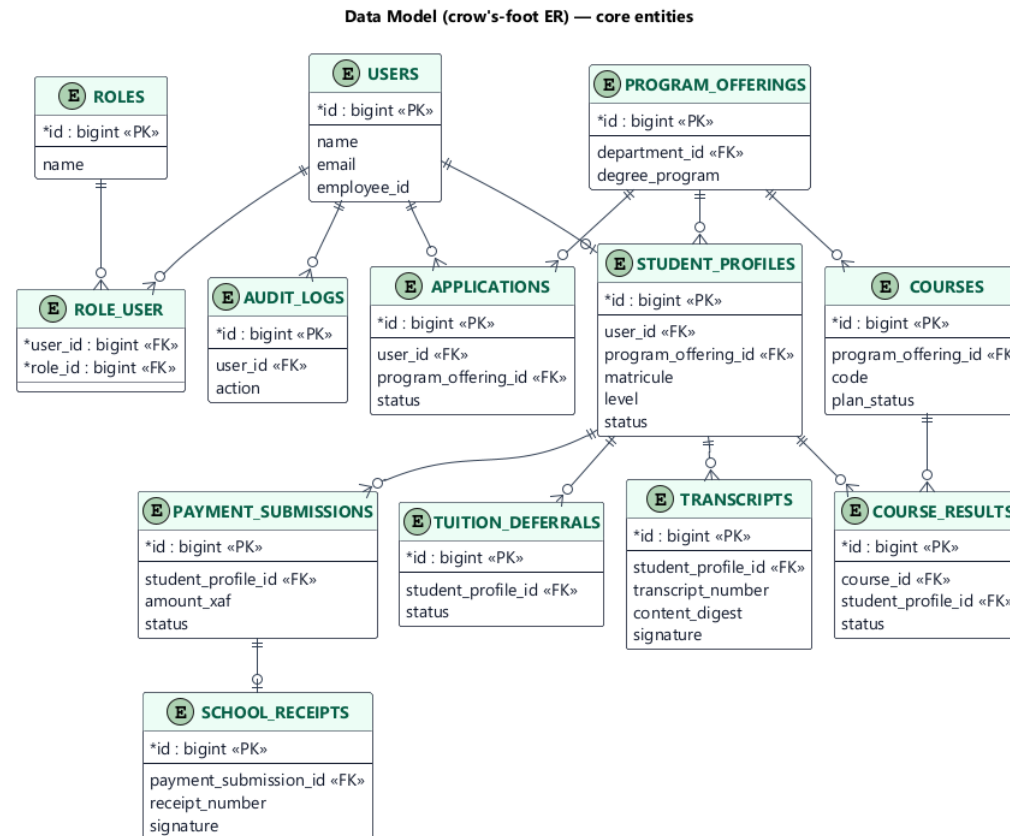
Six roles plus an unauthenticated public verifier.

Domain model (UML class diagram)



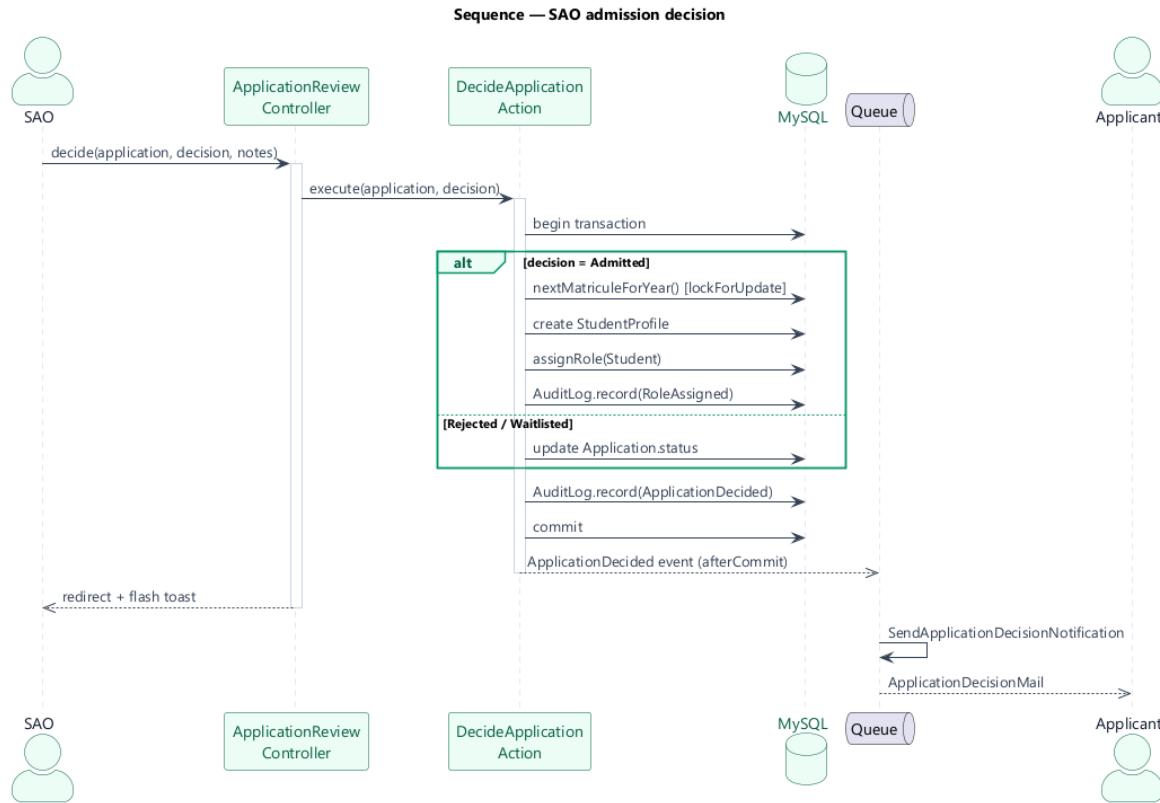
Core classes, attributes, operations and relationships.

Data model (crow's-foot ER)

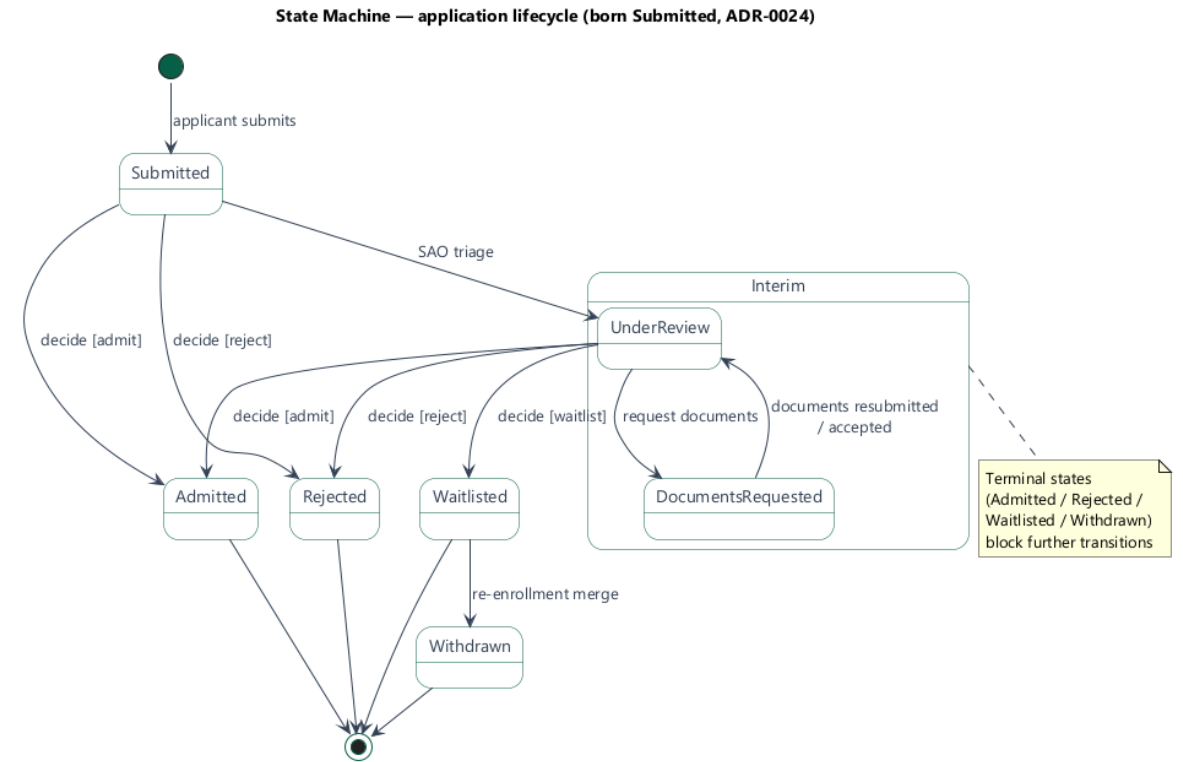


34 migrations; the core entities and their cardinalities.

Admissions — decision & lifecycle

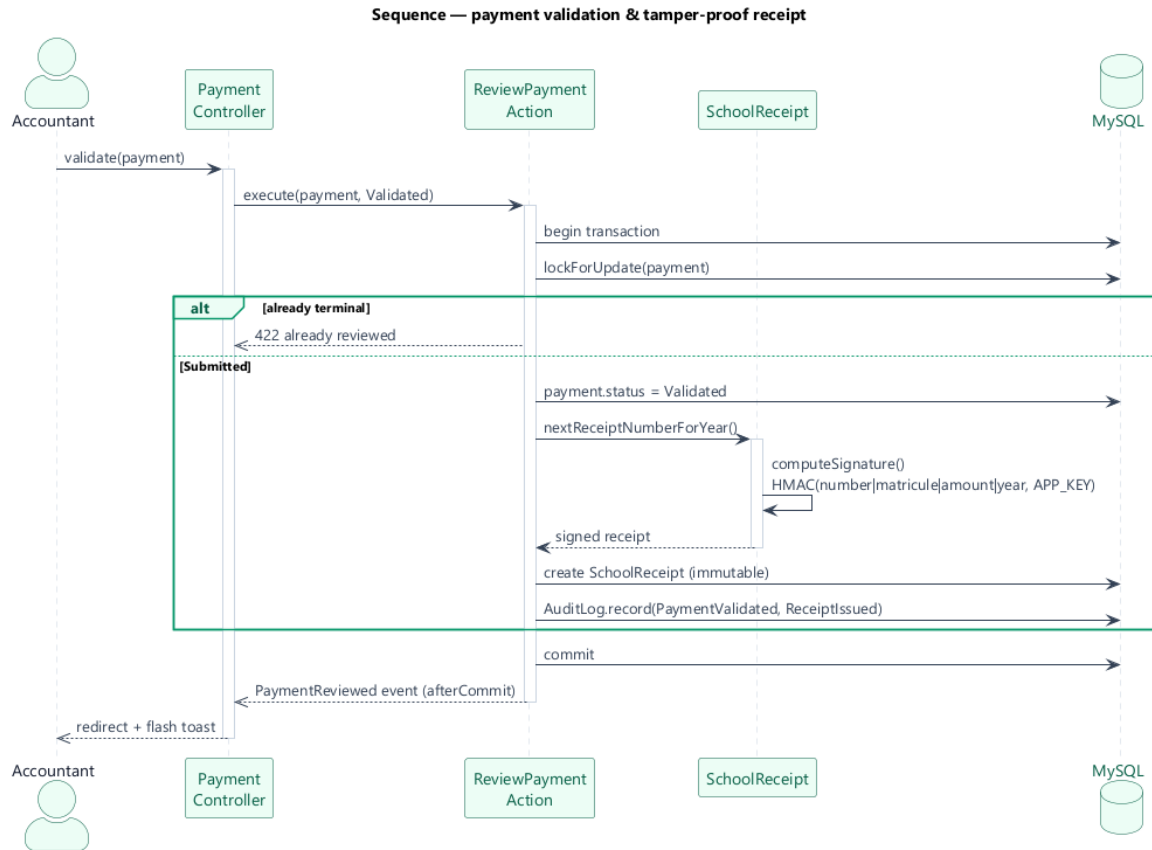


SAO decision sequence (admit mints a student)

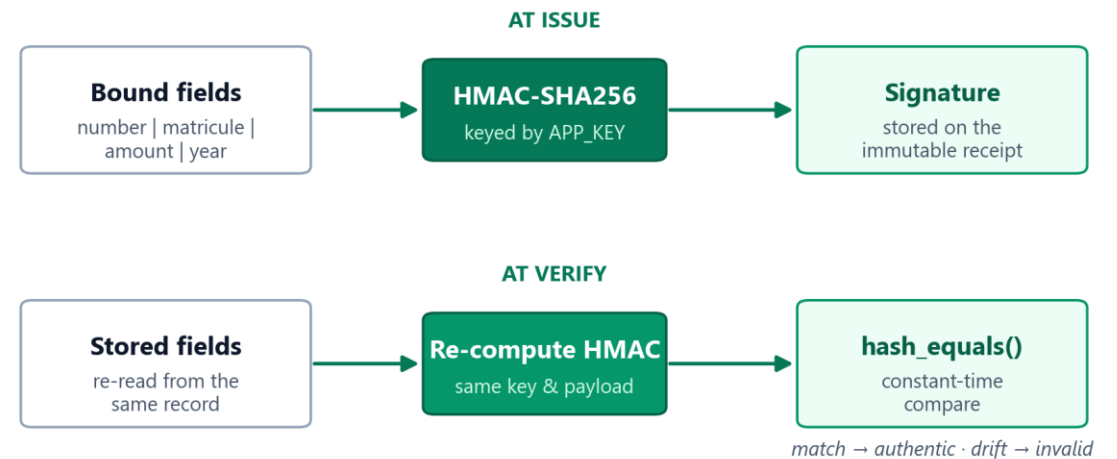


Application state machine (born Submitted)

Tamper-proof payments & receipts

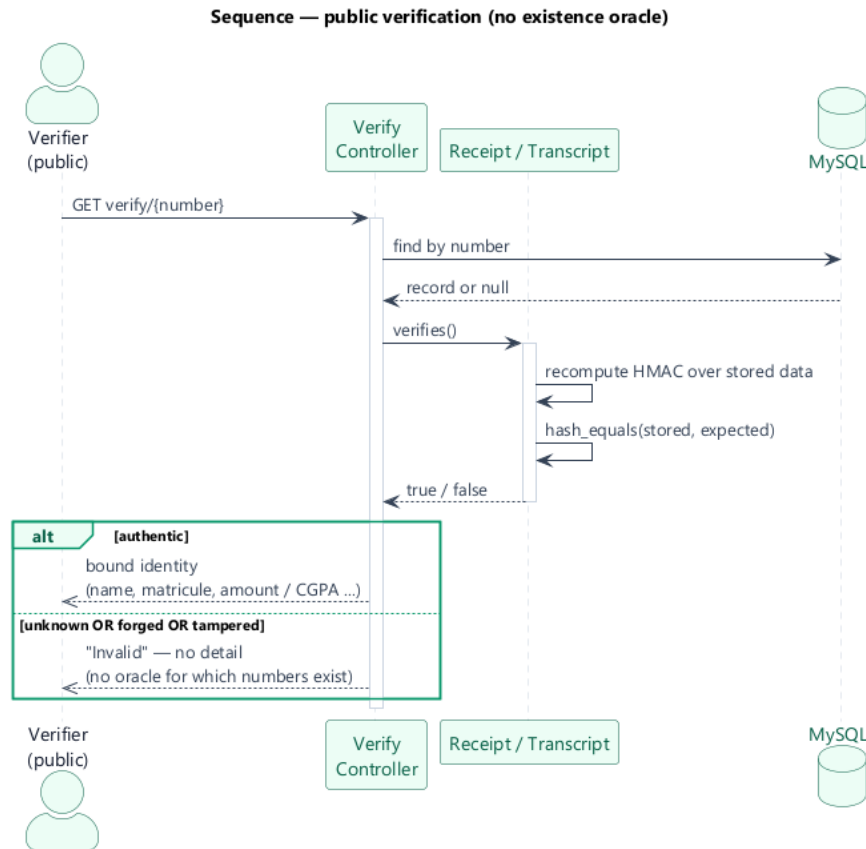


Validate → mint an immutable, signed receipt



HMAC binds identity; verify re-derives it

Verification & the security model

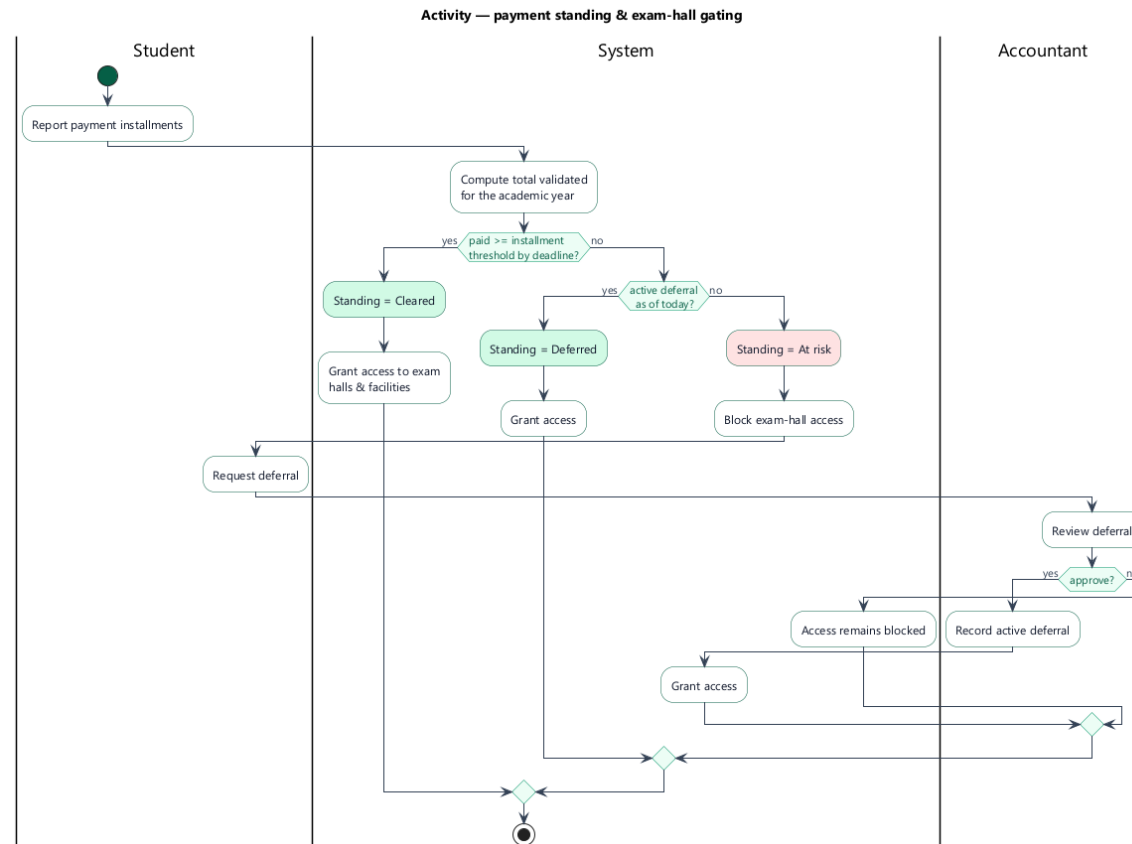


A request must clear every layer

- 1 Session authentication**
Fortify — login, verified email
- 2 Role middleware (role:*)**
coarse gate on every route group
- 3 Per-resource ownership**
you only ever reach your own records
- 4 HMAC-signed, immutable records**
receipts & transcripts can't be forged or altered
- 5 Append-only audit log · no oracle**
every decision traceable; verify leaks nothing

Defense in depth across five layers

Exam gating — standing & deferrals

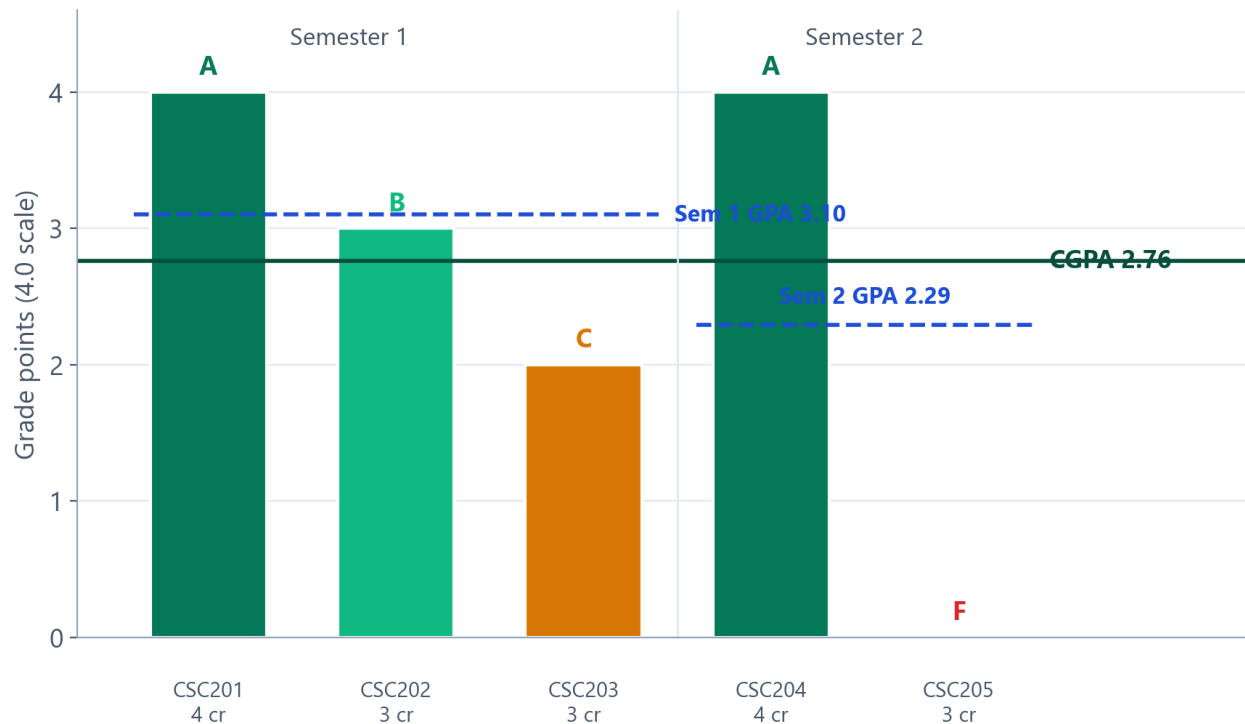


Standing decides exam-hall access each deadline



Below threshold → at risk unless deferred

Academic transcripts (#71)



- **4.0 GPA / CGPA** credit-weighted, per semester & cumulative
- **Immutable snapshot** signed at issue; results may change later
- **mpdf PDF + QR** branded, downloadable transcript
- **Public HMAC verify** authentic-or-invalid, no oracle

RESULTS

Implementation & results

127

routes

34

migrations

28

ADRs

707

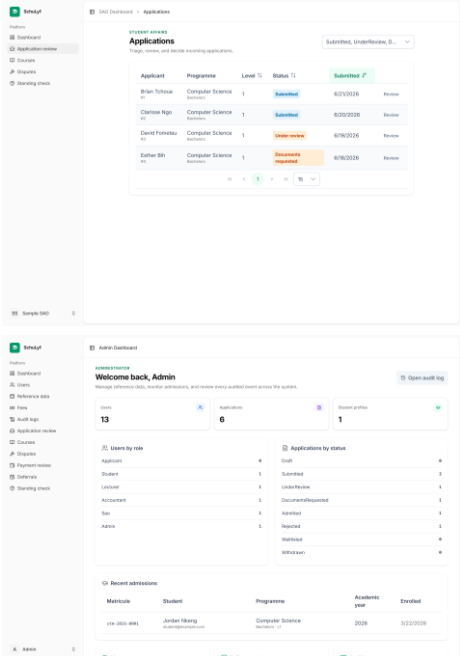
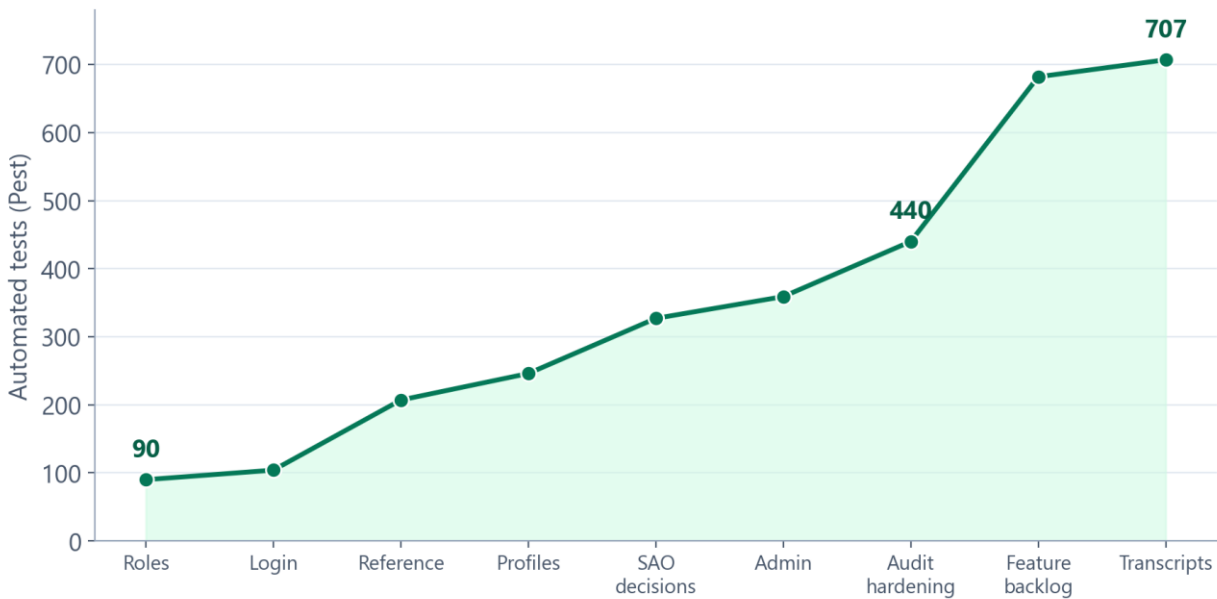
tests

7

modules

6

roles



Real screens — SAO review queue · Admin dashboard

Testing, a hard-won catch & what's next



Every PR — 4 required checks, green before merge

707 tests — the quality gate

Browser · smoke

Feature — integration & HTTP

Unit — pure logic

Engineering rigor

- Test-driven; 707 automated Pest tests
- 4 required CI checks green before every merge
- Per-feature quality gate (Pint · vue-tsc · ESLint · build)

A notable catch

The final whole-branch review found a MySQL JSON key-order bug that made every genuine transcript read “invalid” — invisible to the SQLite test suite. Fixed by canonicalising the digest.

Future work

SMS notification channel · analytics dashboards · mobile app · timetable & scheduling.

Thank you — questions welcome